

Introduction to Geospatial Data Analysis in R

Working with Colorado Climate and Geographic Data

Learning Goals

By the end of this lesson, you will be able to: 1. Load and view maps of Colorado 2. Work with climate data (temperature, rainfall) 3. Extract data for major Colorado cities 4. Create simple maps with colors showing data 5. Understand basic geospatial concepts

Part 1: What is Geospatial Data?

Geospatial data = Data that has location information (latitude, longitude)

Examples: - City locations (Denver: 39.7°N, 104.9°W) - Weather data across Colorado - Population by county - Elevation across the Rocky Mountains

Two main types: 1. **Vector data:** Points, lines, polygons (cities, roads, county boundaries) 2. **Raster data:** Grid of pixels with values (temperature, rainfall maps)

Setup

There are a couple new packages we need to install to work with geospatial data. These are:

- **sf** : working with vector data
- **terra** : working with raster data
- **tmap** : visualizing spatial data (i.e., making maps!)

Install new packages: Only do this ONCE

```
install.packages("sf")
install.packages("terra")
install.packages("mapview")
```

Load in ALL packages for today's lesson:

```
# Install required packages
library(tidyverse)      # For data manipulation
library(sf)             # For vector data (Simple Features)
library(terra)          # For raster data
library(mapview)        # For interactive mapping
```

Part 2: Loading Colorado Map Data

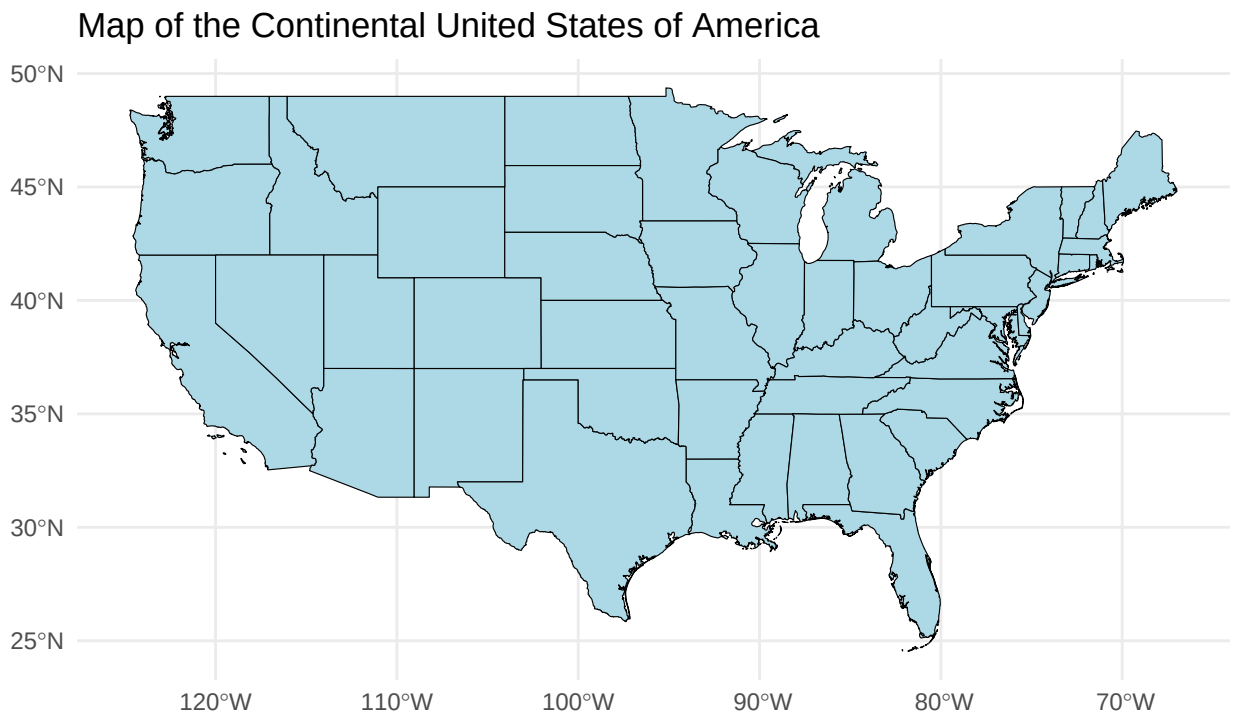
Read in Continental United States boundaries

```
usa <- st_read("data/us_states.shp")

## Reading layer `us_states' from data source
##   `C:\Users\ccmothes\Desktop\sui-be-ess532a\geospatial\data\us_states.shp'
##   using driver `ESRI Shapefile'
## Simple feature collection with 49 features and 121 fields
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -124.7346 ymin: 24.54255 xmax: -66.97732 ymax: 49.36949
## Geodetic CRS:   WGS 84
```

Make a simple map

```
ggplot(usa) +
  geom_sf(fill = "lightblue", color = "black") + # geom_sf lets us plot spatial objects in ggplot
  labs(title = "Map of the Continental United States of America") +
  theme_minimal()
```



Where's Colorado??

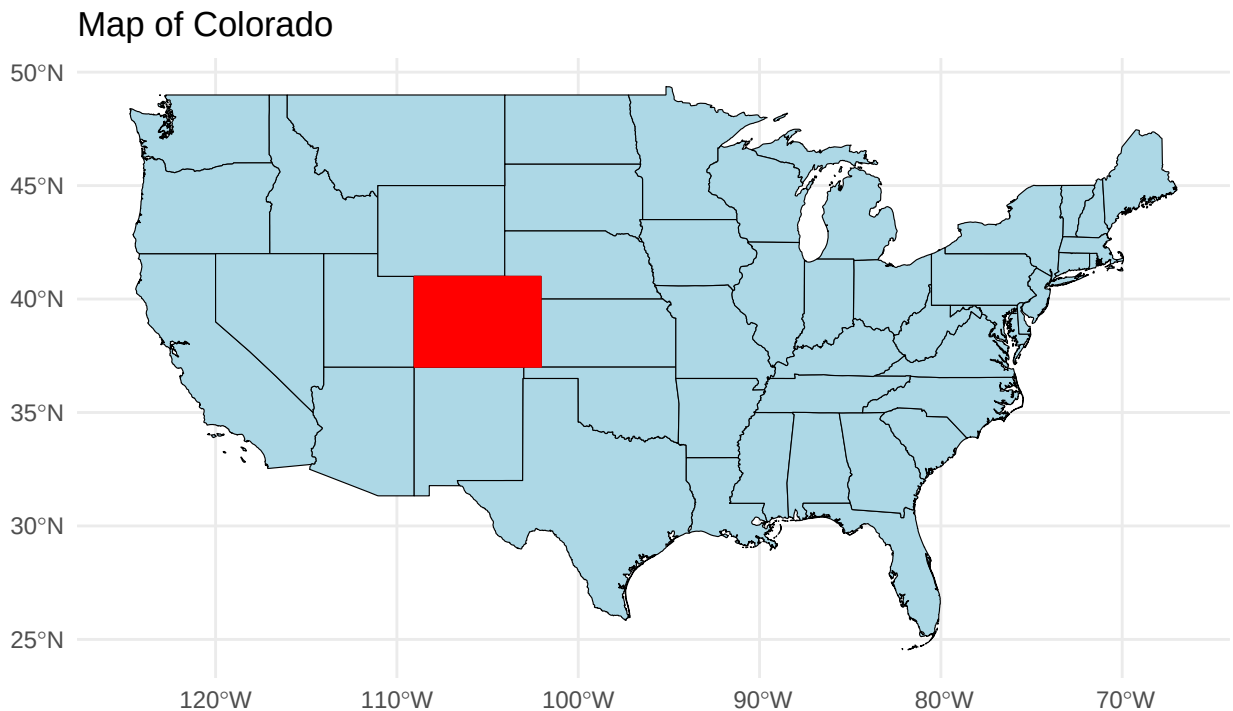
We can actually filter spatial vector objects like data frames

```
colorado <- usa %>% filter(name == "Colorado")
```

Make a simple map

We can use `geom_sf()` to visualize spatial data.

```
# Basic map of Colorado
ggplot(usa) +
  geom_sf(fill = "lightblue", color = "black") +
  geom_sf(data = colorado, fill = "red", color = "Red") +
  labs(title = "Map of Colorado") +
  theme_minimal()
```



Part 3: Working with Colorado Cities

Create dataset of major Colorado cities

```
# Major Colorado cities with coordinates
colorado_cities <- data.frame(
  city_name = c("Denver", "Colorado Springs", "Aurora", "Fort Collins",
    "Lakewood", "Thornton", "Arvada", "Pueblo",
    "Grand Junction", "Boulder"),
  longitude = c(-104.9903, -104.8214, -104.8319, -105.0844,
    -105.0769, -104.9719, -105.0875, -104.6091,
    -108.5506, -105.2705),
  latitude = c(39.7392, 38.8339, 39.7294, 40.5853,
    39.7047, 39.8680, 39.8028, 38.2544,
    39.0639, 40.0150),
```

```

    population_thousands = c(715.5, 478.2, 366.6, 164.2,
                              155.9, 136.2, 118.4, 111.1,
                              63.3, 105.1)
)

```

Creating spatial data

The object we just created above called `colorado_cities` is currently a data frame. In order to visualize and analyze the spatial dataset we need to first convert it to a *spatial* object. When working with the `sf` package these are called *sf* objects.

`st_as_sf()`

We use the `st_as_sf()` to convert a data frame to a spatial *sf* object. There are two main arguments we need to specify in this function: - `coords` =: this specifies which columns in our data frame represent the longitude and latitude. IMPORTANT: When writing this argument the column names must always specify *Longitude first* and *Latitude second*. - `crs` =: this specifies the Coordinate Reference System (CRS) of the data. We will learn more about that later.

```

# Convert to spatial data
cities_sf <- st_as_sf(colorado_cities,
                      coords = c("longitude", "latitude"),
                      crs = 4326) # WGS84 coordinate system

cities_sf

```

```

## Simple feature collection with 10 features and 2 fields
## Geometry type: POINT
## Dimension:      XY
## Bounding box:   xmin: -108.5506 ymin: 38.2544 xmax: -104.6091 ymax: 40.5853
## Geodetic CRS:   WGS 84
##
##      city_name population_thousands      geometry
## 1      Denver              715.5 POINT (-104.9903 39.7392)
## 2 Colorado Springs            478.2 POINT (-104.8214 38.8339)
## 3      Aurora              366.6 POINT (-104.8319 39.7294)
## 4 Fort Collins              164.2 POINT (-105.0844 40.5853)
## 5    Lakewood              155.9 POINT (-105.0769 39.7047)
## 6    Thornton              136.2 POINT (-104.9719 39.868)
## 7    Arvada              118.4 POINT (-105.0875 39.8028)
## 8    Pueblo              111.1 POINT (-104.6091 38.2544)
## 9 Grand Junction           63.3 POINT (-108.5506 39.0639)
## 10 Boulder              105.1 POINT (-105.2705 40.015)

```

Note the 4-digit number we assign for `crs`. This is an EPSG code, which is tied to a specific CRS called WGS84 and one of the most common reference systems coordinates are recorded in (often noted by the fact that the values are in decimal degrees). This is used by Google Earth and all GPS satellites (among others). A full list of EPSG codes and coordinate reference systems can be found [here](#). Note, there are A LOT.

Note on Coordinate Reference Systems

One of the most important part of working with spatial data is the coordinate reference system (CRS) that is used. The CRS describes how and where your spatial data is located on Earth. There are numerous different CRS's depending on when and how the data was collected, the spatial location and extent it was collected, etc. In order to analyze and visualize spatial data, **all objects must be in the exact same CRS**.

We can check a spatial object's CRS by printing the object name to the console, which will return a bunch of metadata about the object. You can specifically return the CRS for `sf` objects with `st_crs()`.

```
st_crs(cities_sf)

## Coordinate Reference System:
##   User input: EPSG:4326
##   wkt:
##   GEOGCRS["WGS 84",
##     ENSEMBLE["World Geodetic System 1984 ensemble",
##       MEMBER["World Geodetic System 1984 (Transit)"],
##       MEMBER["World Geodetic System 1984 (G730)"],
##       MEMBER["World Geodetic System 1984 (G873)"],
##       MEMBER["World Geodetic System 1984 (G1150)"],
##       MEMBER["World Geodetic System 1984 (G1674)"],
##       MEMBER["World Geodetic System 1984 (G1762)"],
##       MEMBER["World Geodetic System 1984 (G2139)"],
##       MEMBER["World Geodetic System 1984 (G2296)"],
##     ELLIPSOID["WGS 84",6378137,298.257223563,
##       LENGTHUNIT["metre",1]],
##     ENSEMBLEACCURACY[2.0]],
##   PRIMEM["Greenwich",0,
##     ANGLEUNIT["degree",0.0174532925199433]],
##   CS[ellipsoidal,2],
##     AXIS["geodetic latitude (Lat)",north,
##       ORDER[1],
##       ANGLEUNIT["degree",0.0174532925199433]],
##     AXIS["geodetic longitude (Lon)",east,
##       ORDER[2],
##       ANGLEUNIT["degree",0.0174532925199433]],
##   USAGE[
##     SCOPE["Horizontal component of 3D system."],
##     AREA["World."],
##     BBOX[-90,-180,90,180]],
##   ID["EPSG",4326]]
```

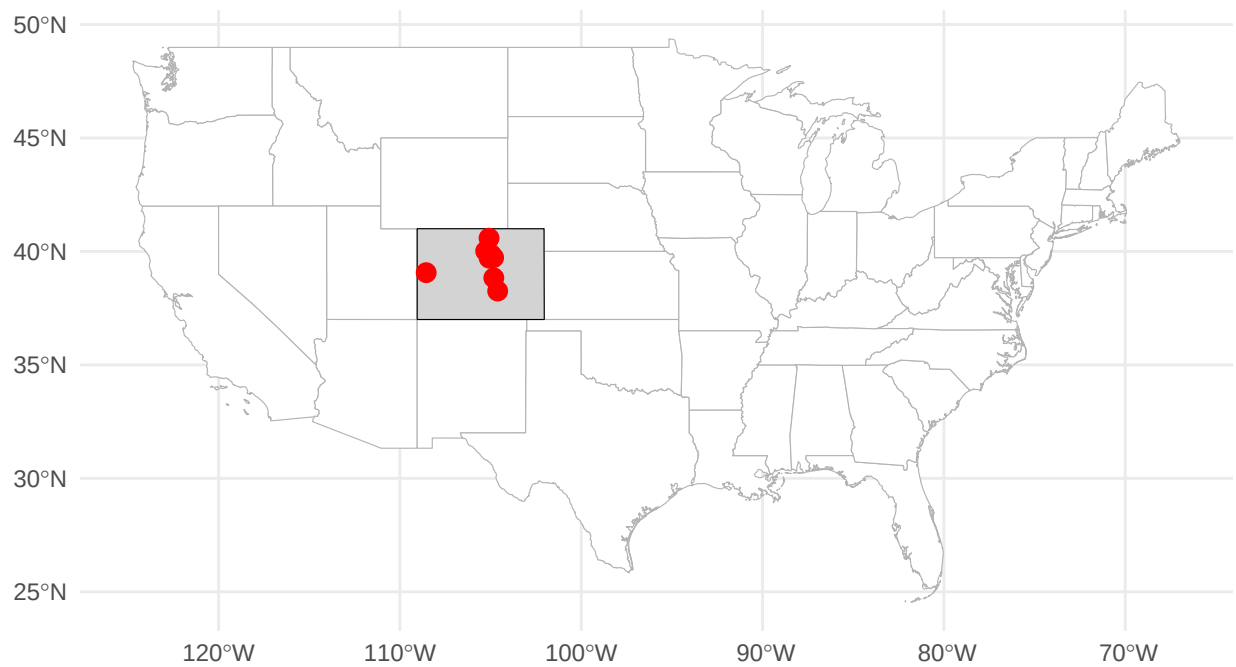
We won't spend too much time with the concept of CRS in this class, but it is very important to know if you plan on working with spatial data in the future.

Map cities on Colorado

Now let's add our city locations to our map of Colorado:

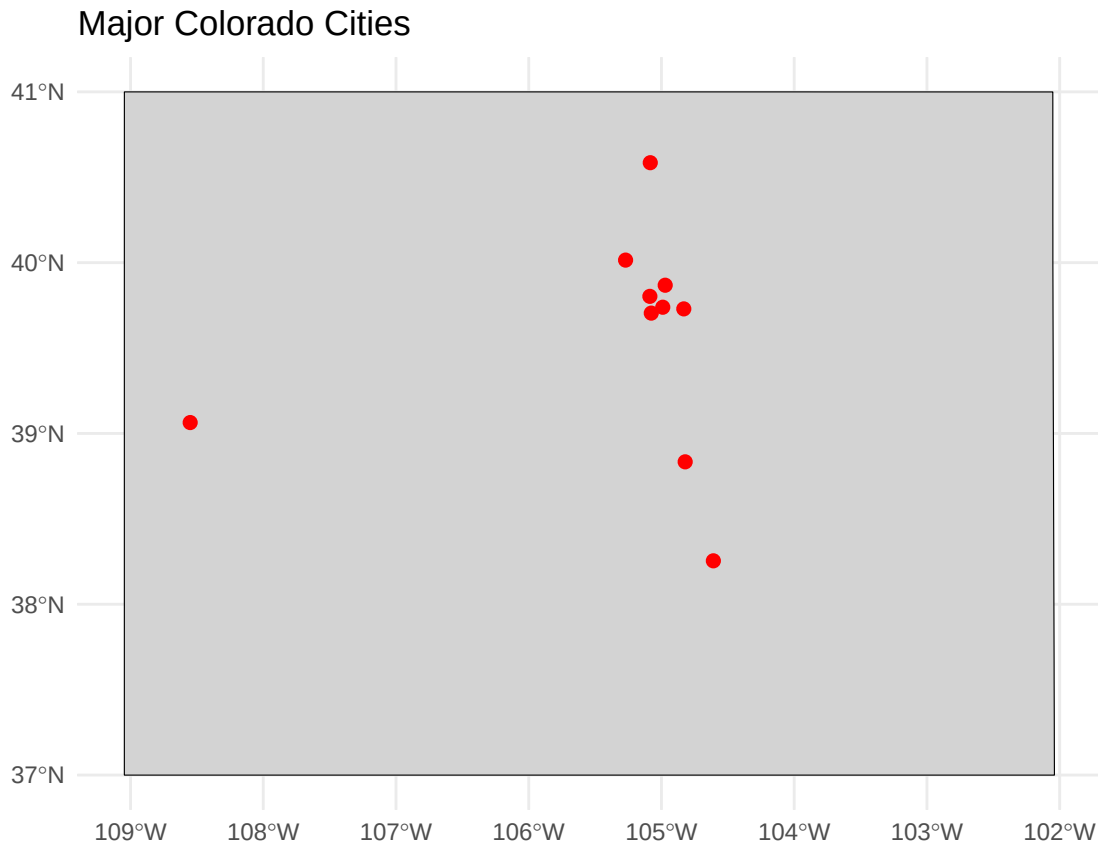
```
ggplot() +
  geom_sf(data = usa, fill = "white", color = "gray70") +
  geom_sf(data = colorado, fill = "lightgray", color = "black") +
  geom_sf(data = cities_sf, color = "red", size = 3) +
  labs(title = "Major Colorado Cities") +
  theme_minimal()
```

Major Colorado Cities



That's hard to see at this zoom level. Now that we know where in the US Colorado is, let's zoom in and just map Colorado:

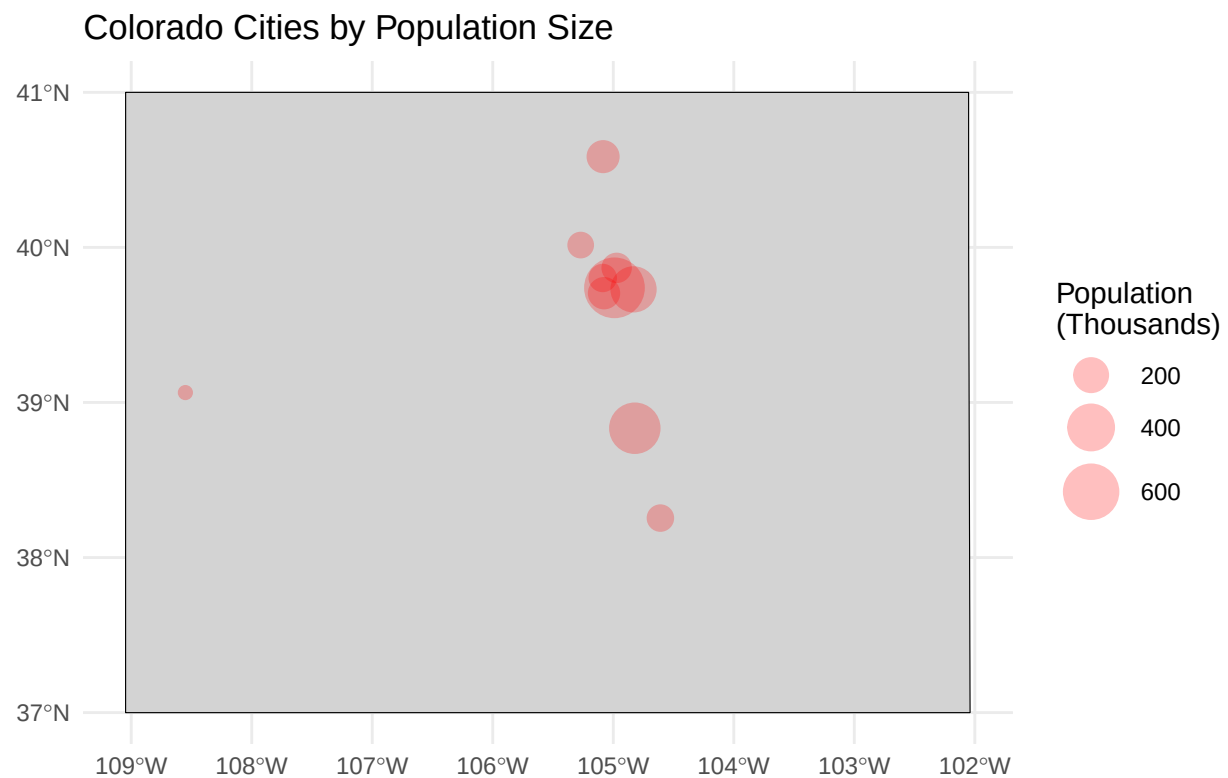
```
ggplot() +  
  geom_sf(data = colorado, fill = "lightgray", color = "black") +  
  geom_sf(data = cities_sf, color = "red", size = 2) +  
  labs(title = "Major Colorado Cities") +  
  theme_minimal()
```



Map with city sizes based on population

Since we have estimated population data as a variable in our `cities_sf` object, we can size the locations by their population:

```
# Cities sized by population
ggplot() +
  geom_sf(data = colorado, fill = "lightgray", color = "black") +
  geom_sf(data = cities_sf, aes(size = population_thousands), # size by population
    color = "red", alpha = 0.25) + # alpha controls the transparency
  scale_size_continuous(name = "Population\n(Thousands)", # name the legend and edit the point size range
    range = c(2, 10)) +
  labs(title = "Colorado Cities by Population Size") +
  theme_minimal()
```



Part 4: Working with Raster Data

Colorado climate data

You have two rasters we have created for you to explore in this lesson. Raster file types often have *.tiff* as the file extension, and we can read them in to R using the `rast()` function from the `terra` package.

```
# Read in precipitation data

## Historic data
co_precip_hist <- rast("data/co_historic_precip.tiff")

## Future data for the year 2050
co_precip_2050 <- rast("data/co_2050_precip.tiff")
```

These two datasets were retrieved from the WorldClim database: <https://www.worldclim.org/>.
 - `co_historic_precip.tiff`: this represents mean annual precipitation from 1970-2000 (mm)
 - `co_2050_precip.tiff`: this represents projected mean annual precipitation for 2041-2060.

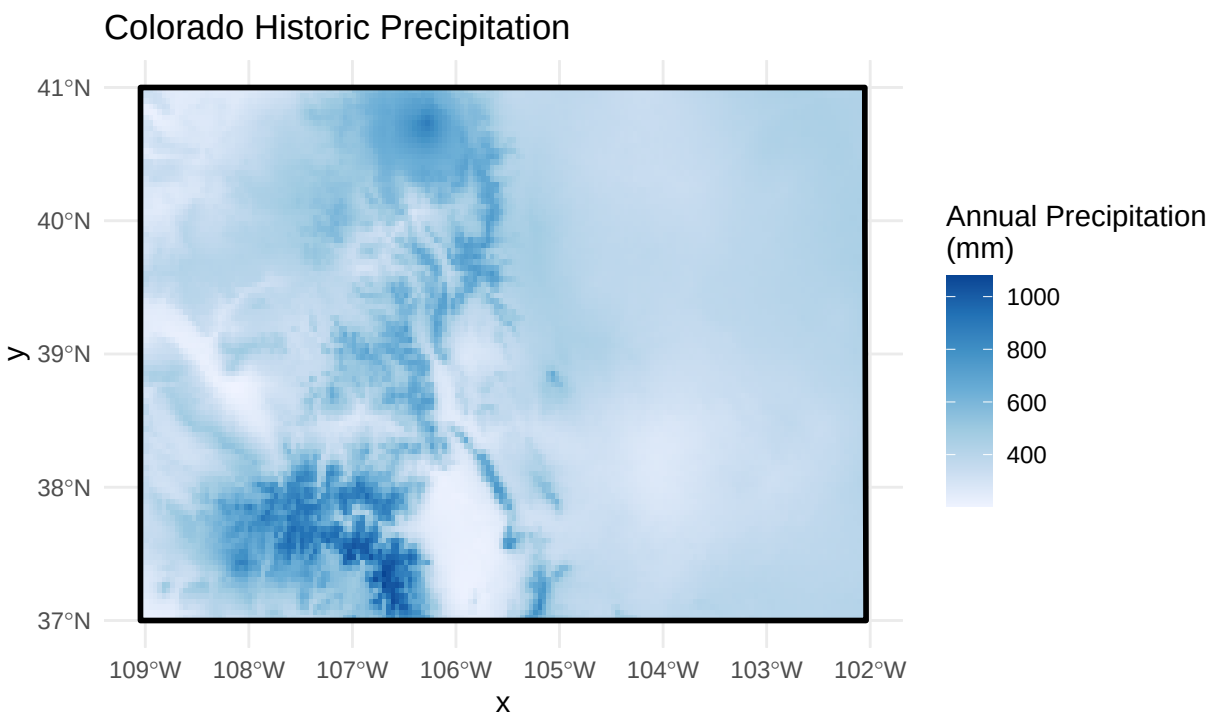
The future dataset represents projected precipitation data for the worst-case scenario SSP5 (Shared Socio-Economic Pathways) from the BCC-CSM2-MR global climate model.

Plot the raster data

To plot a raster in `ggplot()` you have to first convert it to a data frame with the `as.data.frame()` function. Then, use the name of the third column in that new data frame (here it should be `CO_historic_precip`) to assign the `fill =` argument below. Remember tidyverse does not like quotes (" ") around column names.

```
# Convert raster to data frame for ggplot
raster_df <- as.data.frame(co_precip_hist, xy = TRUE)

# Plot raster with state boundary
ggplot() +
  geom_raster(data = raster_df, aes(x = x, y = y, fill = CO_historic_precip)) +
  geom_sf(data = colorado, fill = NA, color = "black", size = 1) +
  scale_fill_distiller(name = "Annual Precipitation\n(mm)", palette = "Blues", direction = 1) +
  labs(title = "Colorado Historic Precipitation") +
  theme_minimal()
```



Part 5: Geospatial Analysis

Raster Calculations

Since we have two rasters for the same region, one showing future precipitation and one showing historic, we can use R to calculate the projected change in precipitation.

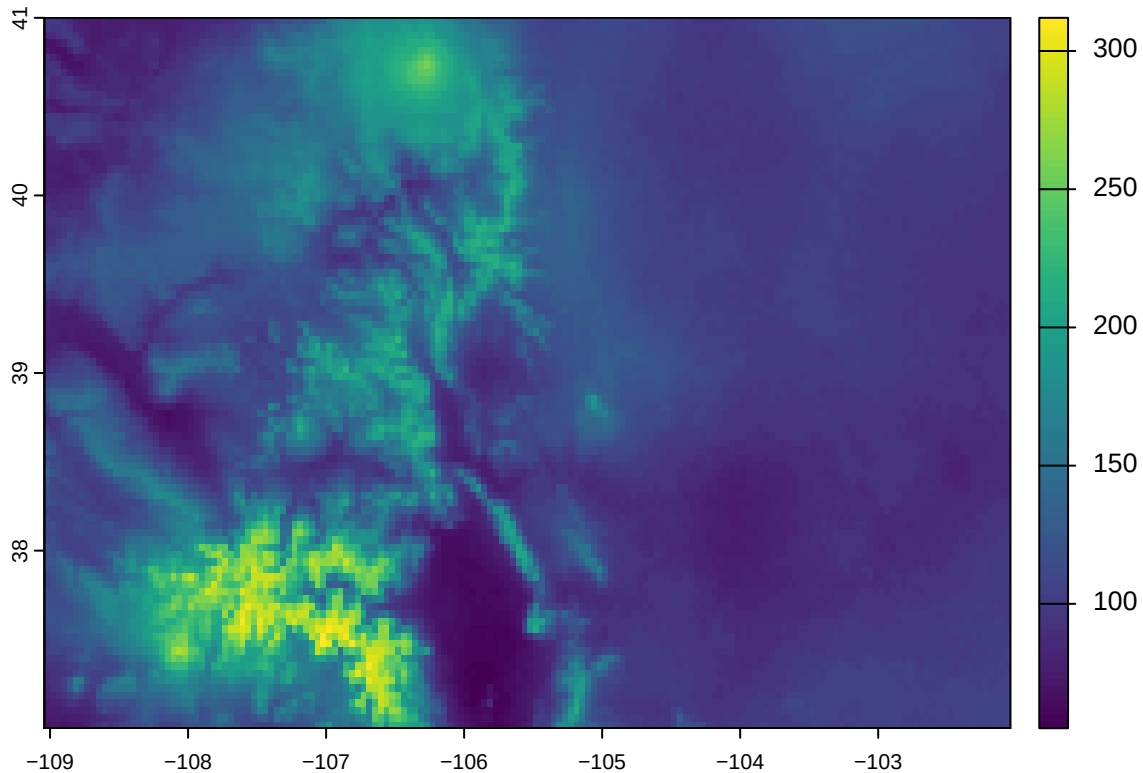
This is surprisingly easy in R with raster math!

Calculate change raster

```
precip_change <- co_precip_2050 - co_precip_hist
```

In addition to `ggplot`, you can very quickly view a basic map using R's base function `plot()`:

```
plot(precip_change)
```



Extract calculations

The change map above is great to see the projected change in precipitation across the entire state, but say we wanted to know the exact projected change for all of our cities to compare amongst them.

We can get the raster values at each of our point locations using the `extract()` function from the `terra` package.

Extract precipitation values at city locations using our `cities_sf` object created above:

```
# Extract raster values at city points  
extracted_precip <- terra::extract(precip_change, cities_sf)
```

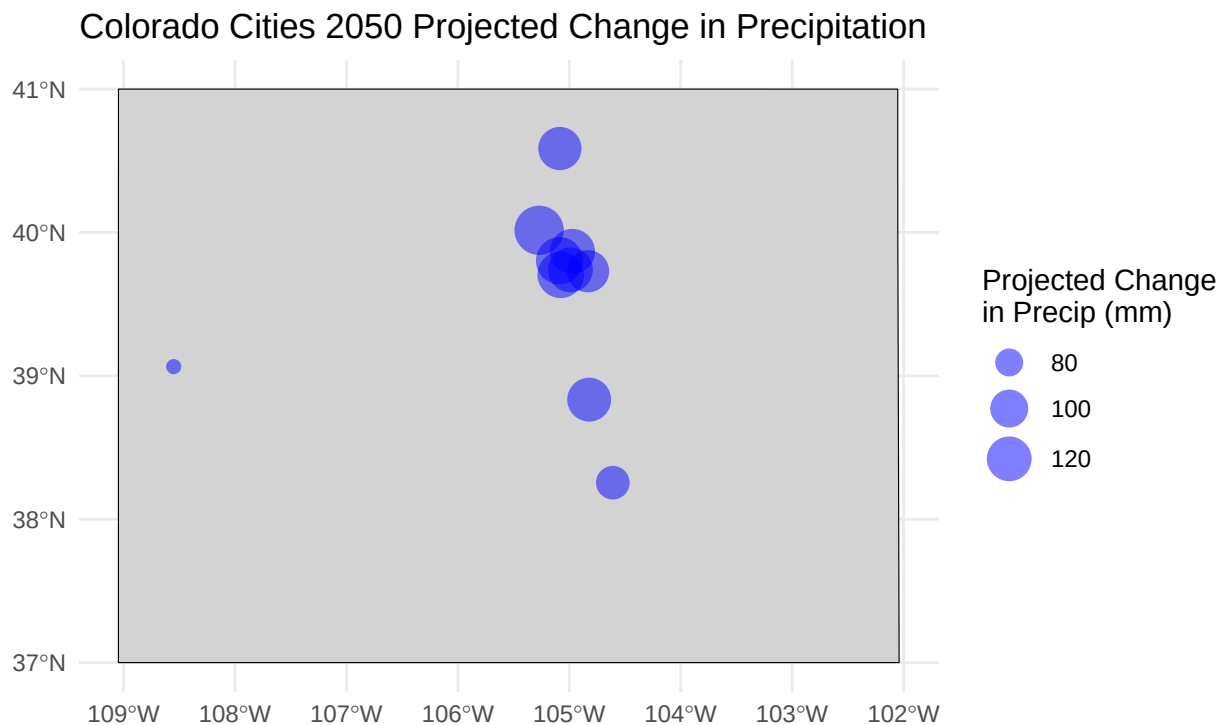
Instead, add it as a new column to our spatial `cities_sf` data using `mutate()`:

Note: Make sure the column name you use with `$` matches the name returned from the `extract()` code we ran above.

```
cities_precip <- cities_sf %>%  
  mutate(precip_change = terra::extract(precip_change, cities_sf)$C0_2050_precip)
```

Now, let's visualize which cities are projected to have the highest change in precipitation:

```
# Cities sized by change in precip
ggplot() +
  geom_sf(data = colorado, fill = "lightgray", color = "black") +
  geom_sf(data = cities_precip, aes(size = precip_change), # size by precip change
          color = "blue", alpha = 0.5) +
  scale_size_continuous(name = "Projected Change\nin Precip (mm)", # name the legend and edit the point
                        range = c(2, 8)) +
  labs(title = "Colorado Cities 2050 Projected Change in Precipitation") +
  theme_minimal()
```



Zonal statistics

Finally, what if we want to know the projected change in precipitation for each county? In this case, we cannot just extract a single value from a single point, but instead we need to summarize multiple values within an entire county.

To summarize a raster within specified regions (polygons) we use what are called zonal statistics. In R, we can run zonal statistics using the `zonal()` function from the `terra` package.

For this function, it also requires we change the format of our `sf` object above. We can easily do this inline using the `vect()` function from `terra`, which will convert the `sf` object to the required format called a “SpatVector” object.

We use the `fun = "mean"` argument to calculate the mean of all values within each county. We also need to add `na.rm = TRUE` so that it ignores any raster cells that may be NA (most likely areas outside the state boundary).

First, let's read in the county boundaries:

```
co_counties <- st_read("data/co_counties.shp")
```

```
## Reading layer `co_counties' from data source
##   `C:\Users\ccmothes\Desktop\sui-be-ess532a\geospatial\data\co_counties.shp'
##   using driver `ESRI Shapefile'
## Simple feature collection with 64 features and 1 field
## Geometry type: MULTIPOLYGON
## Dimension:      XY
## Bounding box:   xmin: -109.0603 ymin: 36.99243 xmax: -102.0415 ymax: 41.00344
## Geodetic CRS:   NAD83
```

```
terra::zonal(precip_change, vect(co_counties), fun = "mean", na.rm = TRUE)
```

```
##      CO_2050_precip
## 1      177.82400
## 2      128.96212
## 3      165.03077
## 4      101.36515
## 5      173.78125
## 6      109.39539
## 7      102.92800
## 8      133.26667
## 9      121.45588
## 10     102.90612
## 11     120.40741
## 12     160.82105
## 13     155.93515
## 14     102.44382
## 15     167.87952
## 16     122.55425
## 17     101.42935
## 18     135.72447
## 19     157.52604
## 20     155.04206
## 21       99.11360
## 22     202.46693
## 23     135.29658
## 24     127.38683
## 25     114.65217
## 26     137.27152
## 27     169.49091
## 28     138.60684
## 29     106.01681
## 30     261.87500
## 31       95.26496
## 32       97.81481
## 33       93.62353
## 34     232.07104
## 35     177.28293
## 36     101.50856
## 37     102.71467
## 38     109.63333
## 39     118.78571
## 40     112.65979
```

```
## 41      104.74576
## 42      152.28571
## 43      106.48571
## 44      101.85195
## 45       82.77778
## 46       87.57754
## 47       67.15094
## 48       89.33333
## 49      124.40000
## 50      115.51506
## 51       92.09259
## 52       80.56452
## 53      127.92405
## 54      135.20603
## 55      142.34064
## 56      103.77076
## 57      127.53057
## 58      150.96774
## 59      111.52941
## 60       97.36480
## 61      141.90299
## 62       88.33779
## 63       98.02262
## 64      234.66165
```

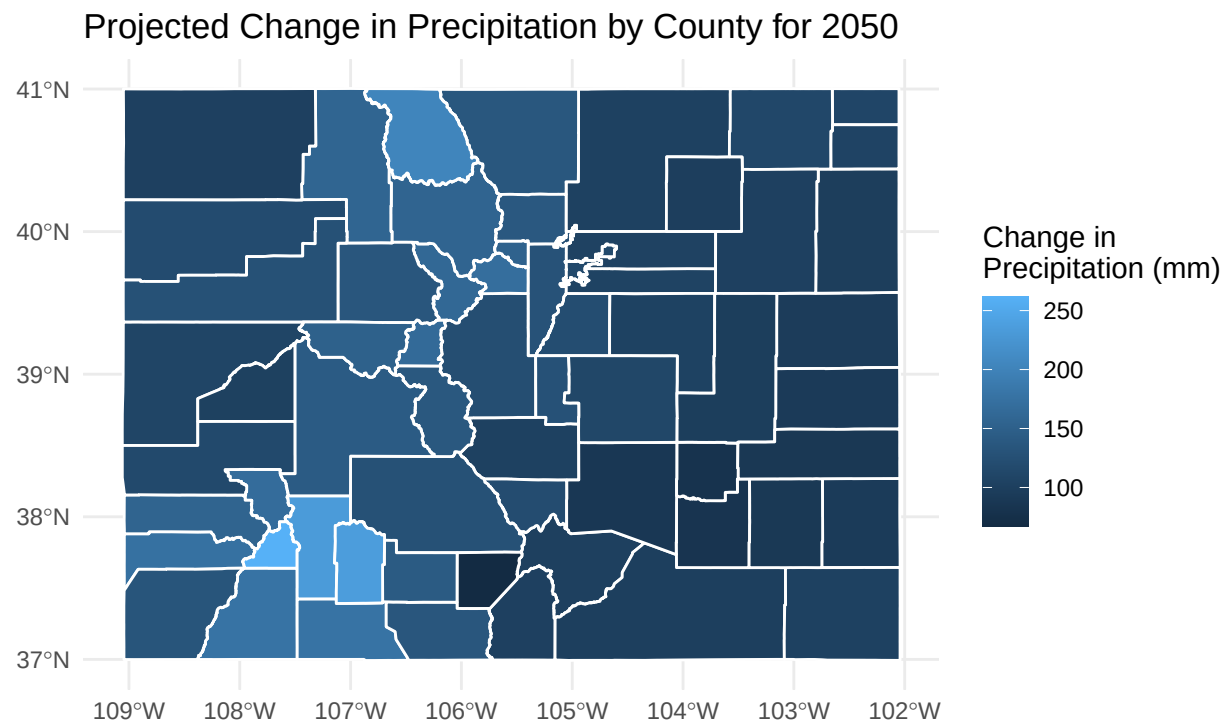
Similar to how we did this with the cities data, we can use `mutate()` to add these calculated means directly to our `co_counties` spatial object as a new variable:

Note: Make sure the column name you use with `$` matches the name returned from the `zonal()` code we ran above.

```
counties_precip <- co_counties %>%
  mutate(precip_change = terra::zonal(precip_change, vect(co_counties), fun = "mean", na.rm = TRUE)$CO_2)
```

Now we can make a map where counties are colored by their projected change in precipitation.

```
# Map showing all counties
ggplot(counties_precip) +
  geom_sf(aes(fill = precip_change), color = "white", size = 0.5) +
  scale_fill_continuous(name = "Change in\nPrecipitation (mm)") +
  labs(title = "Projected Change in Precipitation by County for 2050") +
  theme_minimal()
```



Part 6: Interactive Mapping

The most fun part of maps is being able to interact with them. To do so in R, we have to go beyond `ggplot2` and use a new package you installed earlier called `mapview`.

`mapview` makes interactive maps with a single function call — just pass in any spatial object and it handles the rest. It works with both `sf` vector objects and `terra` rasters.

Map a raster

```
mapview(precip_change)
```

Map a vector layer

```
mapview(cities_precip, zcol = "precip_change")
```

The `zcol =` argument tells `mapview` which column to use for coloring the points — similar to `aes(color = )` in `ggplot`.

Combine multiple layers

You can layer multiple spatial objects using `+`, just like `ggplot`:

```
mapview(precip_change) +  
  mapview(cities_precip, zcol = "precip_change")
```